HS PF

Programmieren in Python

Datentypen

Prof. Dr.-Ing. Stefan Kray

Wiederholung: Grundlegendes (1)

- Whitespace sensitiv (Leerzeichen an falscher Stelle ist Fehler)
- Strukturierung durch Einrückung (Tab=4 Spaces, bitte nicht mischen)
- Python ist objektorientiert
- Automatische Speicherverwaltung → Kein malloc() oder free() wie in C
- Sehr gut lesbar

Wiederholung: Grundlegendes (2)

- Quellcode mit der Endung „.py“
- KEIN Semikolon am Ende
- Intuitiv
- Beispiel: Was gibt folgendes Programm aus:

```
a=-3
if a<0:
    a=5
else:
    a=7
print(a)
```

Elementare Rechenoperationen auf Zahl-Datentypen

Operator	Bedeutung	Beispiel
+	Addition	2+3 (=5)
-	Subtraktion	5-3 (=2)
*	Multiplikation	7*2 (=14)
/	Division	8/3 (=2.6666)
//	Ganzzahlige Division	8//3 (=2)
%	Modulo (Divisionsrest)	5 % 2 (=1)
**	Potenz	2 ** 4 (=16)
&	Bitweises UND	4 & 4 (=4)
|	Bitweises ODER	4 | 3 (=7)
==	auf Gleichheit prüfen	4==4 (=True)
!=	Auf Ungleichheit prüfen	4!=4 (=False)

Datentypen (1)

- In Python sind die Datentypen implizit, d.h. müssen nicht explizit deklariert werden
- Zuweisung: Variable = Wert

- Zahl:
  - Variable „a“ den Wert -3 zuweisen

    ```
    a=-3
    ```
  - Variable „a“ den Wert 3.14 zuweisen

    ```
    a=3.14
    ```
  - Variable „a“ den Wert 3+5j zuweisen

    ```
    a=3+5j
    ```

- String:
- Variable “b“ den Wert „Hallo Welt“ zuweisen: `b="Hallo Welt"`
  - F-String: `b=f"Folgender Wert steckt in a: {a}"`

    → `Folgender Wert steckt in a: (3+5j)`

Datentypen (2)

- Boolean:
  - Variable “a“ den Wert True oder False zuweisen zuweisen

```
a=True
a=False
```

- Listen:

```
a=[1,2,4,5]
a=["a","test","b"]
a=[3.14,"test",-3,True]
a=[3.14,["test",1,-1],-3,True]
```

- Tuple:

```
a=(3.14,"test",-3,True)
```

- Set:

```
a={'a', 'b', 'c', 1, 2, 3}
```

Datentypen (3)

- Dictionary (dict): Paar aus einem Schlüssel und einem Wert

```
a={"Wert":23,42:"Antwort","Liste":[1,2,3]}
```

- None: Kein Wert zugewiesen

```
a=None
```

- File: File-Objekt um Dateien zu lesen oder schreiben

```
f = open("file.txt", "w")
f.write("Text in File speichern.")
f.close()
```

- Abfrage Datentyp mit: `type()`
- Beispiel:

```
a=5
type(a)
```

→ **int**

- **Aufgabe: datentypen.py**

Optionale Deklaration von Datentypen

- „Type hints“ seit Version 3.5; Typenfehler werden im TypeChecker angezeigt
- Syntax:

```
Variable:Type
```

- Funktionsrückgabe mit Pfeil `->`
- Beispiele:

```
alter:int
alter=5
alter="test" #kein Fehler, Typechecker meckert (z.B. PyCharm)



name:str="test"


def f(x:int)->int:
    return x+3
```

Indizierung und Erweitern (1)

- Listen, Tuple, Strings, etc. können indiziert werden
- **Indizierung in Python beginnt bei 0**!
- Beispiel:

```
a="Hallo"
l=[3,6,9]
print(a[0])
print(l[1])
```

  - Ergebnis: H und 6

- Listen (nicht jedoch Tuple) können verändert werden:

```
l=[3,6,9]
l[1]=17
print(l)
```

  - Ergebnis: [3, 17, 9]

Indizierung und Erweitern (2)

- An Listen können Daten angehangen werden mit „append“:

```
l=[] #leere Liste
l=[1,2] #Liste mit 1,2
l.append(3)
print(l)
```

Ergebnis: [1, 2, 3]

geht auch mit „+“

- Listen können vom Ende aus indiziert werden mit negativen Indizes:

```
l=[1,2]
print(l[-1])
```

Ergebnis: 2

```
print(l[-2])
```

Ergebnis: 1

Indizierung und Erweitern (3)

- Slicing: Auf Teile einer Liste zugreifen

  `[start:ende]` → Elemente mit Index start bis Index ende aus Liste kopieren

  `[start:ende:schrittweite]` → andere Schrittweite nehmen

```
l=[1,2,3,4,5,6,7,8,9]
print(l[1:3])
print(l[1:8:2])
```

- Statt Notation mit Doppelpunkt auch per Funktion „slice“ möglich:

```
print(l[slice(1,8,2)])
```

- Sublisten: Logik analog zu normalen Listen

```
l=[[1,2,3],[4,5,6],[7,8,9]]
print(l[1])
print(l[1][1:])
```

Rechnen mit Datentypen

- mit Zahlen kann gerechnet werden, z.B. 1+3**2
- Auf Strings kann Elementweise zugegriffen werden:

```
a="Hallo"
print(a[1]) #was gibt das aus?
```

- String können nicht elementweise geändert werden

- Auf Tupel kann ebenfalls elementweise zugegriffen werden
- Tupel können jedoch nicht geändert werden

```
l=(1,3,5)
print(l[2])
```

- Länge einer Liste / eines Tupels / eines Strings mit dem Befehl „len“:

```
len(l) #ergibt 3
```

Rechnen mit Datentypen (2)

- Rechnen mit Variablen
  - Addition, Subtraktion, Multiplikation, Division, Potenz
- Länge von Strings / Listen
- Zuweisung, Ersetzen von Listen
- Listen anhängen (l+l)
- etc.

Aufgabe „mittelwert.py“

- Erstellen Sie ein Programm „mittelwert.py“
  - Lesen Sie 5 Zahlen des Benutzers mit dem Befehl **input()** ein.
  - Hinweis: input() liefert einen String zurück. Mit dem Befehl **int(input())** erhalten Sie eine Zahl.
  - Erstellen Sie eine Liste l aus den 5 Zahlen des Benutzers
  - Berechnen Sie den Mittelwert der Liste und geben Sie das Ergebnis aus.

Weitere Funktionen und Operatoren (1)

- auf Gleichheit prüfen: ==
- Ungleich: !=
- Größer als: >
- Größer gleich: >=
- Kleiner als: <
- Kleiner gleich: <=

- Prüfen ob Element in Liste oder String: „in“

```
42 in [1,13,42,99]
```

→ **True**

Weitere Funktionen und Operatoren (2)

- in Datentypen konvertieren:
  - int()
  - str()
  - float()
  - bool()
  - list()

- negieren:
  - not

- Addition:
  - auf für Listen: `[1,2,3]+[4,5,6]`
    → Listen zusammenfügen
  - Strings: `"abc"+"def"`
    → `"abcdef"`

Weitere Funktionen und Operatoren (3)

- Betrag bilden: `abs(a)`
- Maximum zweier Zahlen: `max(a,b)`
- Minimum zweier Zahlen: `min(a,b)`
- Zahlen runden: `round(a)`
- Benutzereingabe: `input("Eingabe bitte:")`
- Liste sortieren: `sorted(l)`
- Summe von Zahlen einer Liste: `sum`(l)
- … zahlreiche weitere Funktionen, siehe https://docs.python.org/3/library/functions.html

  - **Mittelwert einfacher mit sum() zu realisieren**

Iteratoren (1)

- Neben den Listen gibt es noch einen weiteren wichtigen Datentyp: Iteratoren
- Iteratoren sind „nicht ausgerechnete Listen“
- Äquivalent in C++: std::iterator
- Iteratoren können mit dem Befehl „list“ in Listen umgewandelt werden
- Iteratoren liefern einen Wert nach dem anderen. Aufruf mit „next()“.
- Erst mit dem Aufruf von next() wird der NÄCHSTE WERT des Iterators ausgerechnet.
- Beispiel: die Funktion reversed() liefert eine Rückwärts sortierte Liste als Iterator zurück.

```
l=[1,2,3,4,5,6]
li=reversed(l)
print(next(li))
print(next(li))
print(next(li))
```

  - **Ergebnis: 6, 5, 4**

Iteratoren (2)

- Beispiel: map
- Die Funktion „map“ wendet eine Funktion auf eine Liste an. Sie gibt dann einen Iterator zurück.
- Beispiel:

```
li=map(round,[1.1,2.2,3.3,4.4])
print(next(li))
print(next(li))
```

  - **Ergebnis: 1 , 2**

- Will man nur eine Liste zurückhaben, kann man auch direkt eine Transformation in eine Liste anwenden:

```
list(map(round,[1.1,2.2,3.3,4.4]))
```

  Ergebnis: [1,2,3,4]

Range

- Weiterer wichtiger Typ: „Range“
- Zahlenbereich, ähnlich zu Liste
- Wird für Schleifen benutzt
- Beispiel:

```
list(range(0,5))
```

Ergebnis: [0, 1, 2, 3, 4]

- Dritter Parameter in Range: Schrittweite

```
list(range(2,8,2))
```

Ergebnis: [2, 4, 6]

Aufgabe: 7 Tage Inzidenz berechnen

- Laden Sie das Skript „corona_manuell.py“
- Die Liste `ncases` enthält die Inzidenzwerte für Deutschland seit dem 22.01.2020 für jeden Tag.
- Schreiben Sie ein Programm, dass die 7 Tage Inzidenz der letzten 7 Tage berechnet

  Formel:

  $$\text{Inzidenz} = \frac{\#Tag1 + \#Tag2 + \#Tag3 + \#Tag4 + \#Tag5 + \#Tag6 + \#Tag7}{Einwohnerzahl} * 100.000$$